from their users' perspectives as early as possible, ultimately improving the user experience.

2.5.3 Lack of buy-in

Contract testing requires a lot of collaboration across different teams. Therefore, getting everyone to contribute might be more difficult because it takes time to see the value of contract testing. To mitigate this, it's important that you address concerns, provide training and support, and create an environment that encourages open communication and shared responsibility.

As we mentioned, contract testing requires at least two teams—the consumer and the provider. If only one team is invested, then contract testing won't succeed. To avoid this situation, implement our suggestions from section 2.4. Overcoming any resistance to change and fostering a culture of collaboration and trust are essential for successful adoption.

2.5.4 More technical than other testing activities

Based on our experience, establishing a contract testing framework demands additional technical skills compared to writing integration or E2E tests. Contract tests are run like traditional unit tests, which are usually engineered by developers. Contract testing tools, such as Pact, often entail a steeper learning curve compared to other test automation frameworks. To fully harness the benefits of contract testing, it's necessary to grasp and implement key Pact concepts such as can-i-deploy, webhooks, and versioning, among others. In chapter 8, we'll guide you through the implementation of these advanced concepts, ensuring a comprehensive understanding of their usage and advantages.

Setting up a contract testing framework with tools such as Pact requires a solid understanding of the underlying concepts and practices specific to contract testing. While writing integration or E2E tests primarily revolves around interacting with the application interfaces, contract testing focuses on defining and validating the contracts between consumer and provider